



API RESTful

¿Qué es API?

Application Programming Interface

Interfaz de programación de aplicaciones

Una API es un puente que conecta diversos tipos de software o aplicaciones y puede crearse en varios lenguajes de programación.

Las APIs son fundamentales en el mundo de la programación y la tecnología, ya que permiten que diferentes sistemas, aplicaciones o servicios se comuniquen entre sí y compartan datos de manera eficiente y segura.

Una API debe tener una documentación clara y objetiva para poder facilitar su implementación y su uso.

Se utiliza un formato predefinido de datos para compartir información entre los sistemas con el objetivo de lograr la integración entre ellos.

Los más usados son:

- XML (Extensible Markup Language),
- YAML (YAML Ain't Markup Language)
- JSON (JavaScript Object Notation) para las aplicaciones web.

Arquitecturas APIs

REST y SOAP

Son dos estilos arquitectónicos comunes para implementar APIs.

- **REST** (Transferencia de Estado Representacional) utiliza el protocolo HTTP para realizar operaciones CRUD (Crear, Leer, Actualizar, Borrar) sobre recursos.
- **SOAP** (Simple Object Access Protocol) es un protocolo más complejo que utiliza XML para definir mensajes y puede emplear otros protocolos además de HTTP.

A background image showing several black pencils with colored erasers (blue, green, yellow, pink) pointing towards a central point, creating a starburst effect.

Arquitectura REST

Es una arquitectura para desarrollar software, fue definida por Roy Fielding en el año 2000, uno de los principales artífices de la especificación del protocolo HTTP.

¿REST vs RESTful?

- API REST (Interfaz de Programación de Aplicaciones basada en REST):

- Se refiere a cualquier interfaz de programación de aplicaciones que siga los principios de la arquitectura REST.
- Puede **no seguir todas las restricciones** y principios de REST de manera estricta.
- En una API REST, se pueden implementar algunas operaciones sin cumplir con todos los estándares y restricciones definidos por la arquitectura REST.

- API RESTful (API que sigue los principios REST):

- Se refiere a una API que sigue fielmente los principios de la arquitectura REST.
- Cumple con las restricciones y características principales de REST, como la utilización de URIs (Identificadores de Recursos Uniformes) para identificar recursos, el uso de métodos HTTP (GET, POST, PUT, DELETE) para operar sobre esos recursos, el intercambio de representaciones de recursos a través de formatos como JSON o XML, y la ausencia de estado en la comunicación cliente-servidor.
- Una API RESTful se adhiere estrictamente a los principios definidos por REST y, por lo tanto, se considera una implementación más completa y correcta de la arquitectura REST.

¿REST vs RESTful?

- REST viene de **RE**presentational **St**ate **T**ransfer
- Es un tipo de arquitectura de desarrollo web que se apoya totalmente en el estándar HTTP.
- REST se compone de una lista de reglas que se deben cumplir en el diseño de la arquitectura de una API.
- Las API web que funcionan con las limitaciones de arquitectura REST se llaman **API de RESTful** o **Servicios Web RESTful**.
- Muchas veces se usan como sinónimos (REST y RESTful).

API RESTful

<https://github.com/akabab/starwars-api>

<https://swapi.dev/documentation>

<https://rickandmortyapi.com/>

Arquitectura REST

1. **Arquitectura Cliente-Servidor y Recursos:** La arquitectura REST sigue el modelo cliente-servidor, donde los sistemas se dividen en clientes que realizan solicitudes y servidores que procesan esas solicitudes y ofrecen recursos. Los recursos, identificados por URIs (Identificadores de Recursos Uniformes), son la representación de la información que puede ser accedida o modificada a través de la interfaz REST.
2. **Protocolo Cliente/Servidor sin Estado:** En REST, cada solicitud HTTP contiene toda la información necesaria para ser procesada por el servidor. Esto implica que ni el cliente ni el servidor deben mantener un estado previo entre las solicitudes. No se guardan sesiones en el servidor, lo que hace que las interacciones sean independientes y autocontenidas.
3. **Uso Correcto de HTTP:** La arquitectura REST utiliza los métodos HTTP de manera explícita y coherente. Los verbos estándar de HTTP como GET (para obtener datos), POST (para enviar datos), PUT (para actualizar recursos), DELETE (para eliminar recursos) y otros, se utilizan para operar sobre los recursos.
4. **Interfaz Uniforme o Identificador Único (URI):** Cada recurso en una API REST debe tener un identificador único (URI). Estos identificadores deben ser únicos y representar de manera lógica la jerarquía de los recursos. Es esencial mantener una estructura lógica y coherente en la asignación de URIs para los recursos.

Arquitectura REST

5. **Capacidad de Almacenamiento en Caché:** REST aprovecha la capacidad de almacenamiento en caché proporcionada por el protocolo HTTP. Utiliza cabeceras de respuesta que permiten que los recursos se almacenen en la caché, lo que mejora la eficiencia y la velocidad al permitir a los clientes almacenar en caché las respuestas para usos posteriores.
6. **Sistema de Capas:** La arquitectura REST permite la inclusión de servidores intermediarios o capas entre el cliente y el servidor final. Estas capas pueden ser utilizadas para aumentar la escalabilidad, implementar seguridad adicional o proporcionar características de rendimiento como balanceo de carga o almacenamiento en caché, sin que el cliente tenga que preocuparse por los detalles internos de implementación.

Arquitectura REST

Códigos de estado

En la arquitectura RESTful, el envío de códigos de estado HTTP es un aspecto esencial para comunicar el resultado de las solicitudes. Estos códigos de estado son mensajes estándar que indican el resultado de una solicitud HTTP.

En una API RESTful bien diseñada, el uso adecuado de estos códigos de estado permite a los clientes comprender el resultado de sus solicitudes de manera clara y consistente.

Estos códigos de estado también son útiles para los desarrolladores al identificar y solucionar problemas con las solicitudes o el servidor.

Arquitectura REST y Códigos de estado

Códigos de Éxito (2xx):

- 200 OK:** La solicitud ha tenido éxito.
- 201 Created:** Se ha creado un nuevo recurso según la solicitud.
- 202 Accepted:** La solicitud ha sido aceptada para procesamiento, pero puede que no se haya completado.
- 204 No Content:** La solicitud se ha completado con éxito, pero no hay contenido para devolver.

Errores del Cliente (4xx):

- 400 Bad Request:** La solicitud del cliente es incorrecta o no se puede procesar.
- 401 Unauthorized:** Se requiere autenticación para acceder al recurso, pero el cliente no ha proporcionado credenciales válidas.
- 403 Forbidden:** El servidor entiende la solicitud, pero el cliente no tiene permiso para acceder al recurso.
- 404 Not Found:** El recurso solicitado no se ha encontrado en el servidor.
- 405 Method Not Allowed:** indicar que el método de solicitud utilizado (GET, POST, PUT, DELETE, etc.) no está permitido para el recurso solicitado.
- **406 Not Acceptable:** Indica que el servidor no puede producir una respuesta que coincida con la lista de valores aceptables definidos en las cabeceras del cliente, por ejemplo, pide xml y el recurso genera JSON.

Errores del Servidor (5xx):

- 500 Internal Server Error:** Indica un error en el servidor que impide que se complete la solicitud.
- 503 Service Unavailable:** El servidor no está disponible temporalmente debido a una sobrecarga o mantenimiento.
- 504 Gateway Timeout:** El servidor actuó como un proxy y no recibió una respuesta a tiempo desde un servidor ascendente.

Códigos de estado HTTP: representan el resultado

- RESTful exige que sea enviado un código de estado HTTP informando del resultado de la petición.
- Se utilizan los códigos de estado de http
- Por ejemplo, si realizamos un request de POST deberemos devolver un 201.
- Se pueden producir múltiples errores en la llamada al API:
 - falta de permisos
 - errores de validación
 - incluso un error interno de servidor.
- Siempre se debe devolver un código de estado HTTP con los requests.
- Para enviar la respuesta en PHP debemos utilizar un **header**, con el código correspondiente, ejemplo:
`header ("HTTP/1.1 200 OK");`

1XX Informational		4XX Client Error Continued	
100	Continue	409	Conflict
101	Switching Protocols	410	Gone
102	Processing	411	Length Required
2XX Success		412	Precondition Failed
200	OK	413	Payload Too Large
201	Created	414	Request-URI Too Long
202	Accepted	415	Unsupported Media Type
203	Non-authoritative Information	416	Requested Range Not Satisfiable
204	No Content	417	Expectation Failed
205	Reset Content	418	I'm a teapot
206	Partial Content	421	Misdirected Request
207	Multi-Status	422	Unprocessable Entity
208	Already Reported	423	Locked
226	IM Used	424	Failed Dependency
3XX Redirection		426	Upgrade Required
300	Multiple Choices	428	Precondition Required
301	Moved Permanently	429	Too Many Requests
302	Found	431	Request Header Fields Too Large
303	See Other	444	Connection Closed Without Response
304	Not Modified	451	Unavailable For Legal Reasons
305	Use Proxy	499	Client Closed Request
307	Temporary Redirect	5XX Server Error	
308	Permanent Redirect	500	Internal Server Error
4XX Client Error		501	Not Implemented
400	Bad Request	502	Bad Gateway
401	Unauthorized	503	Service Unavailable
402	Payment Required	504	Gateway Timeout
403	Forbidden	505	HTTP Version Not Supported
404	Not Found	506	Variant Also Negotiates
405	Method Not Allowed	507	Insufficient Storage
406	Not Acceptable	508	Loop Detected
407	Proxy Authentication Required	510	Not Extended
408	Request Timeout	511	Network Authentication Required
		599	Network Connect Timeout Error

Códigos de estado HTTP: representan el resultado

RESTful exige que sea enviado un código de estado HTTP informando del resultado de la petición.

Para enviar la respuesta en PHP debemos utilizar un header, con el código correspondiente, ejemplo:

```
header ("HTTP/1.1 200 OK");
```

Códigos de estado HTTP: representan el resultado

Resultados de GET y DELETE:

- 200 OK HTTP/1.1
- 404 NOT FOUND HTTP/1.1

```
header("HTTP/1.0 200 Ok");
```

```
header("HTTP/1.0 404 Not Found");
```

```
header("HTTP/1.0 204 No Content");
```


Códigos de estado HTTP representan el resultado

Resultados de POST y PUT

- 201 (Recurso creado correctamente)
- ¿Qué URL tiene el recurso recién creado?
 - La convención en REST es devolver la URL en la respuesta, como valor de la cabecera HTTP Location
- 400 (petición incorrecta, p.ej. falta un campo o su valor no es válido)

```
header("HTTP/1.0 201 Created");
```

```
header("HTTP/1.0 400 Bad Request");
```

Códigos de estado HTTP: representan el resultado

405 Método no permitido El método HTTP de la solicitud no se permite en el recurso.

```
header("HTTP/1.0 405 Method Not Allowed");
```

406 Not Acceptable Indica que el servidor no puede producir una respuesta que coincida con la lista de valores aceptables definidos en las cabeceras del cliente, por ejemplo, pide xml y el recurso genera JSON.

```
header("HTTP/1.0 406 Not Acceptable");
```

500 Error interno del servidor Se produjo un error interno del servidor al procesar la solicitud. Por ejemplo, al conectar con la Base de Datos.

```
header("HTTP/1.0 500 Internal Server Error");
```

403 Prohibido, cuando intentas acceder a un recurso para el que no tienes permisos

```
header("HTTP/1.0 403 Forbbiden");
```

C
L
I
E
N
T
S



USERS



APPLICATION
OR PROGRAMS

HTTP GET

/ALLCONTACTS

HTTP PUT

/UPDATECONTACT/8472

OUR CLIENTS
SEND HTTP REQUESTS
& WAIT FOR RESPONSES

REQUEST
CLIENT CAN BE USERS OR PROGRAMS ASKING
FOR THE INFORMATION THEY NEED AND
WAITING FOR THE RESPONSE

S
E
R
V
E
R

RESTFUL API

RECEIVES HTTP REQUESTS FROM
CLIENTS & PERFORMS WHATEVER THE
REQUEST NEEDS

FOR EXAMPLE:

- RETRIEVE CONTACT LIST
- RETRIEVE SINGLE CONTACT
- ADD CONTACT
- UPDATE CONTACT DETAILS
- DELETE EXISTING CONTACT

Visual
LANSA

HTTP VERB

HTTP DEFINES A SET OF REQUEST METHODS
TO INDICATE THE ACTION TO BE PERFORMED
FOR A GIVEN RESOURCE. ALTHOUGH THEY
CAN ALSO BE NOUNS, REQUEST METHODS
ARE SOMETIMES REFERRED AS HTTP VERBS

TYPICAL HTTP VERBS

GET → READ FROM DATABASE / FILE SYSTEM
PUT → UPDATE/REPLACE ROW IN DATABASE
PATCH → UPDATE / MODIFY ROW IN DATABASE
POST → CREATE NEW RECORD IN THE DATABASE
DELETE → DELETE FROM THE DATABASE

DATABASE



QUERY

DATABASE

QUERY

FILE SYSTEM



DOCUMENTS | IMAGES

OUR REST API CAN QUERY THE DATABASE OR
RETURN DOCUMENTS/IMAGES BASED ON
INFORMATION NEEDS

RESPONSE

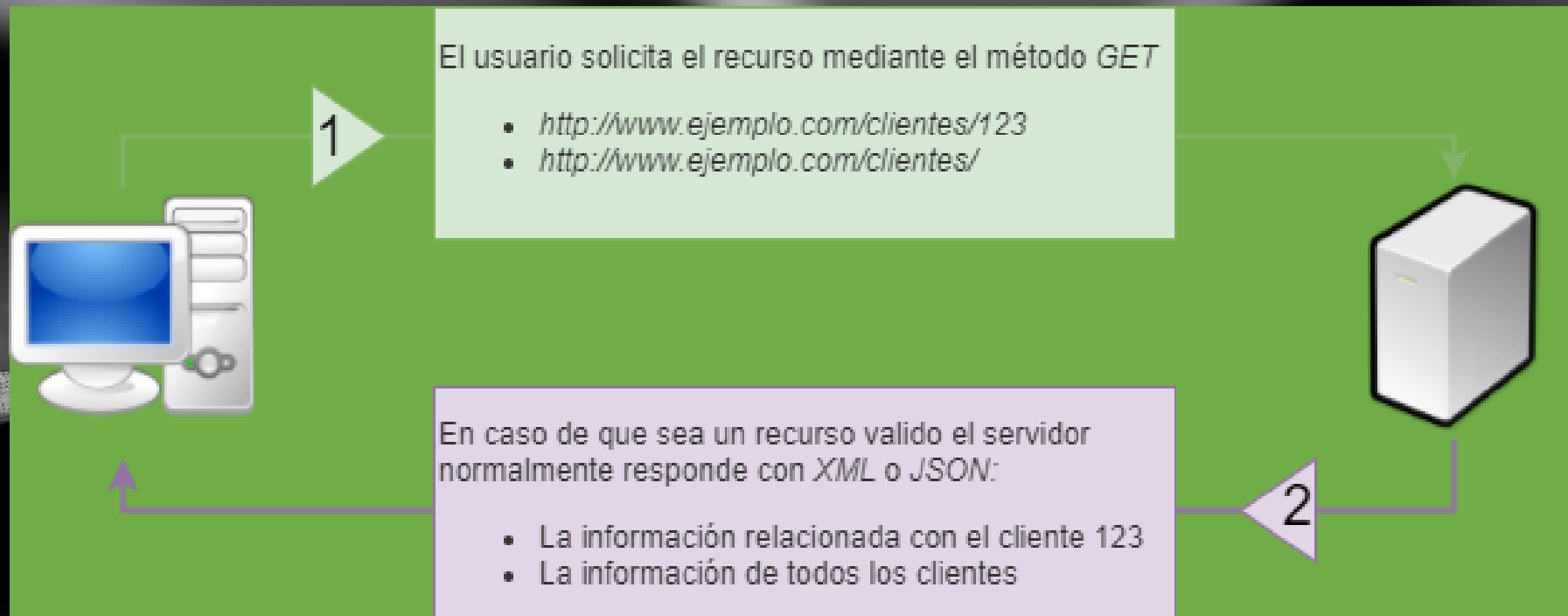
WHEN THE REST API HAS WHAT IT NEEDS, IT
SENDS BACK A RESPONSE TO THE CLIENTS.
THIS WOULD TYPICALLY BE IN JSON OR XML
FORMAT

¿Cómo funciona una API REST?

- El cliente no necesita saber qué procedimiento se llama en el servidor.
- En su lugar, utiliza un conjunto de comandos (llamados “verbos”) que están integrados en HTTP y cuando el comando llega al otro extremo, depende del sistema receptor saber qué hacer con él.
- Por ejemplo, el verbo HTTP que se usa para recuperar datos es “GET”.
- El sistema cliente (llamado “*consumidor*”) y el sistema servidor (el “*proveedor*”) son tan independientes entre sí, que pueden usar diferentes lenguajes (***Java, Python, Ruby, PHP, etc.***) para su implementación general.

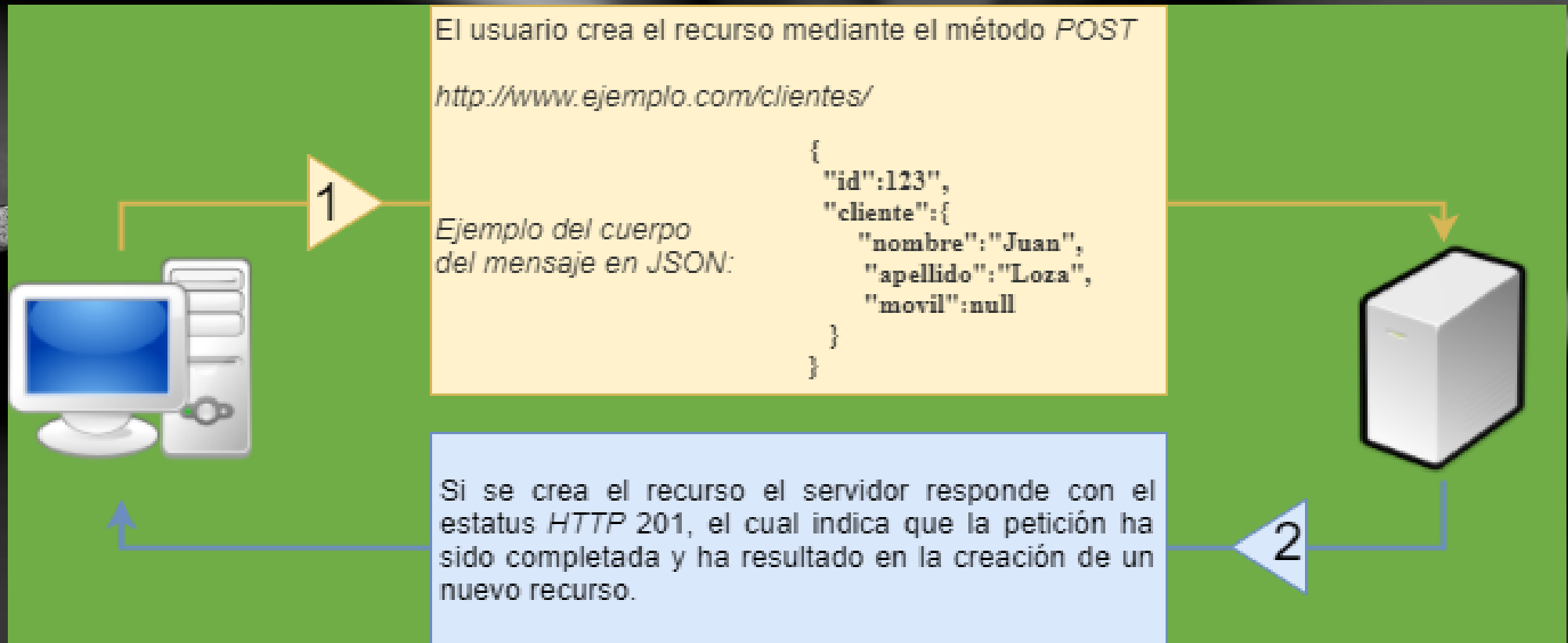
Operación	Método HTTP	URI	Parámetros	Resultado
Listar	GET	/ {recurso}	No aplica	Lista del tipo de recurso, archivo JSON
Crear	POST	/ {recurso}	Archivo JSON	Se crea un nuevo recurso
Leer	GET	/ {recurso}/ {recurso_id}	No aplica	Recurso en función al id
Actualizar	PUT	/ {recurso}/ {recurso_id}	Archivo JSON	Se actualiza el recurso
Borrar	DELETE	/ {recurso}/ {recurso_id}	No aplica	Se elimina el recurso en función al id

GET

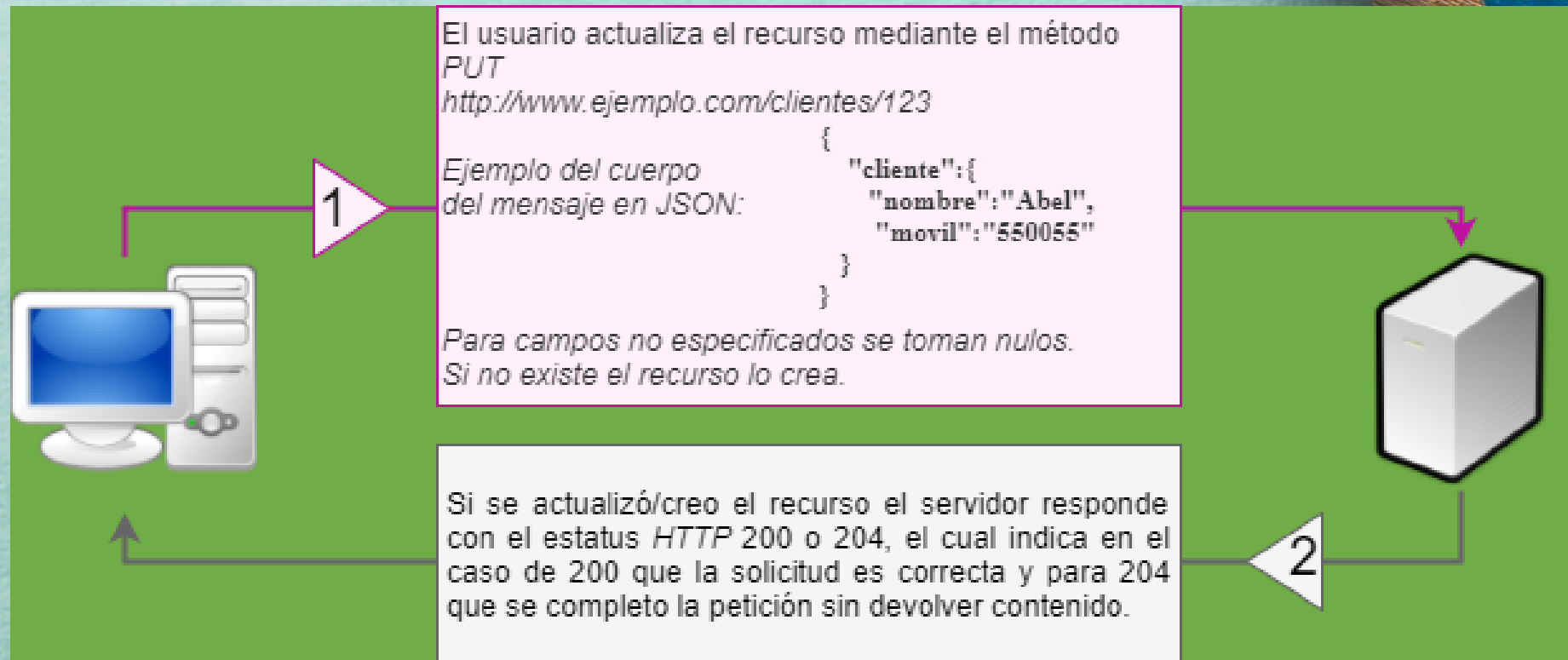


El código de estado HTTP representa el resultado:
200 OK HTTP/1.1
404 NOT FOUND HTTP/1.1

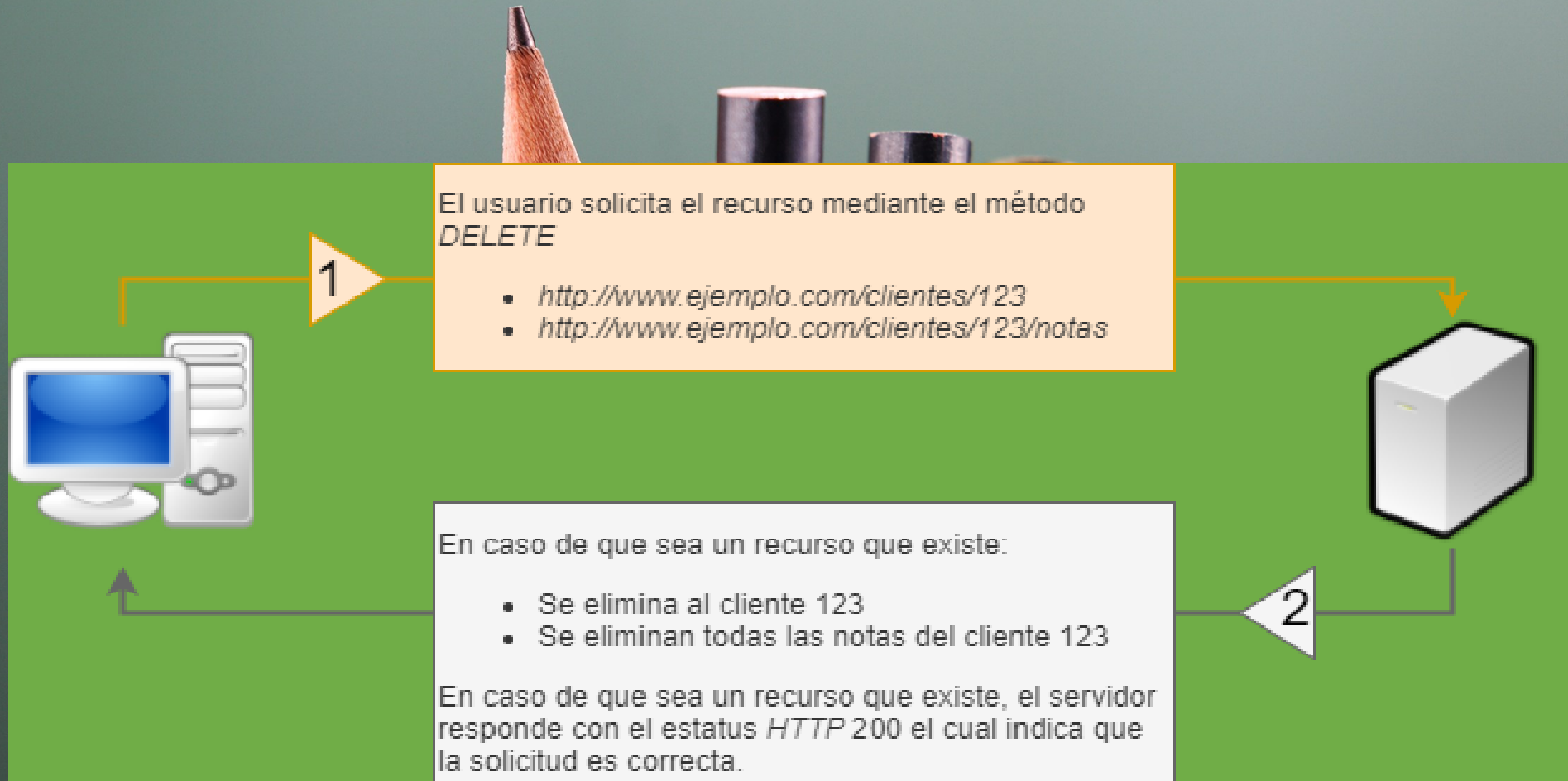
POST



PUT



DELETE



Interfaz Uniforme

Endpoint: Punto de acceso a un recurso

Uso correcto de URIs

Uso correcto de URIs

Endpoint

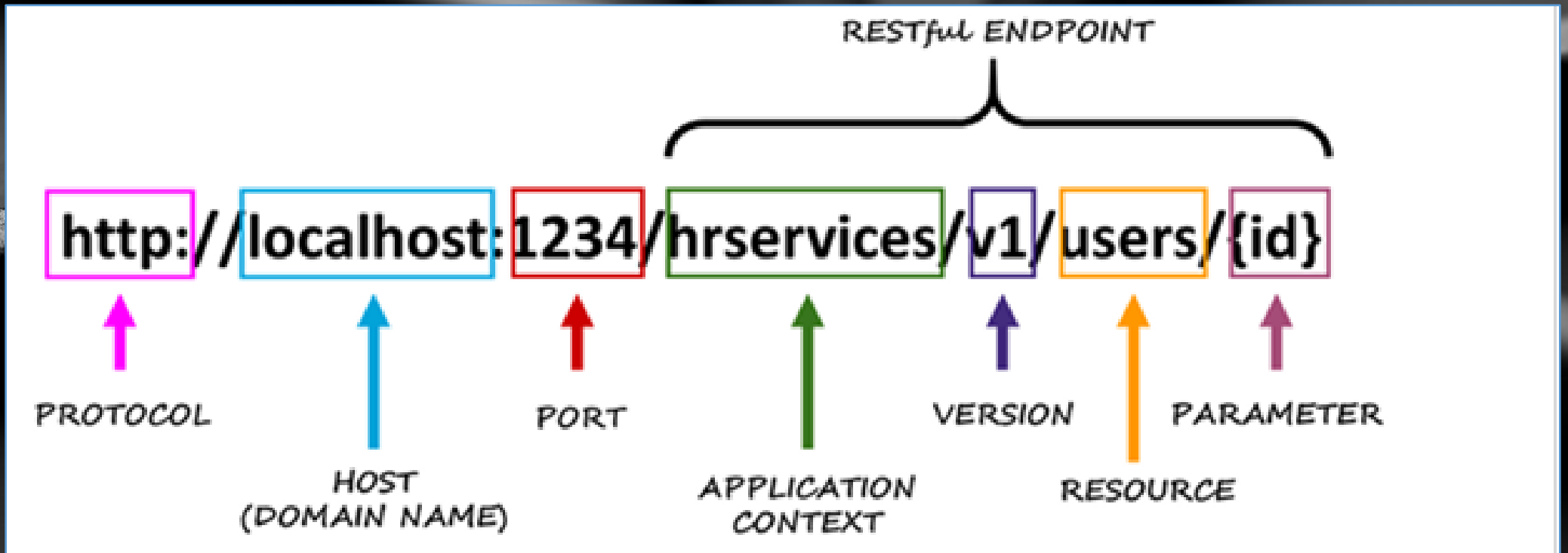
- Cuando desarrollamos una aplicación web, las URLs nos permiten acceder a cada uno de las páginas, secciones o documentos del sitio web.
- Cada página, información, archivo, cuando hablamos de REST, los nombramos como **recursos**.
- El recurso por lo tanto es la información a la que queremos acceder o que queremos modificar o borrar, **independientemente de su formato**.
- Las URL, Uniform Resource Locator , son un tipo de URI, Uniform Resource Identifier, que además de permitir **identificar de forma única el recurso**, nos permite localizarlo para poder acceder a él o compartir su contenido.

Estructura de una URI

protocolo://dominio o hostname: puerto (opcional)/ruta hasta el recurso
/nombre_recurso?**{consulta_de_filtrado}**

Uso correcto de URIs

Endpoint (punto de acceso a un recurso)



Uso correcto de URIs

Endpoint (punto de acceso a un recurso)

- Los URIs de los servicios APIS REST deberían ser intuitivos hasta el punto en que sean fáciles de adivinar.
- Define URIs como estructuras de directorio.

Uso correcto de URIs

Endpoint (punto de acceso a un recurso)

- Obtener todos los usuarios:
 - Método: GET
 - Endpoint: /usuarios
- Obtener un usuario específico:
 - Método: GET
 - Endpoint: /usuarios/123
- Crear un nuevo usuario:
 - Método: POST
 - Endpoint: /usuarios
- Actualizar un usuario existente:
 - Método: PUT
 - Endpoint: /usuarios/123
- Eliminar un usuario:
 - Método: DELETE
 - Endpoint: /usuarios/123

Uso correcto de URIs

Endpoint

Existen varias reglas básicas para ponerle nombre a la URI de un recurso:

- Los nombres de URI **no** deben implicar una acción, por lo tanto, debe evitarse usar **verbos** en ellos.
- Deben ser únicas, no debemos tener más de una URI para identificar un mismo recurso.
- Deben ser independiente de formato.(Archivos sin extensión)
- Deben mantener una jerarquía lógica.
- Los **filtrados de información de un recurso no se hacen en la URI**, sino con una cadena de consulta (**string query**).

Uso correcto de URIs

Endpoint (punto de acceso a un recurso)

Buenas prácticas al crear las URIs:

- Mantener todo en letra minúscula.
- Sustituir los espacios con guiones o con subrayados (elegir un tipo).
- Evitar consultar las cadenas de caracteres tanto como se pueda.

Uso correcto de URIs

Endpoint (punto de acceso a un recurso)

Las URIs no deben implicar acciones y deben ser únicas

- Por ejemplo, la URI */facturas/234/editar* sería incorrecta ya que tenemos el verbo editar en la misma.
- Para el recurso factura con el identificador 234, la siguiente URI sería la correcta, independientemente de que vayamos a editarla, borrarla o actualizarla (la acción la indicamos a través de (GET, DELETE o PUT):

/facturas/234

Uso correcto de URIs

Endpoint (punto de acceso a un recurso)

Las URIs deben ser independientes de formato

- Por ejemplo, la URI **/facturas/234.php** no sería una URI correcta, ya que estamos indicando la extensión php en la misma.
- Para el recurso factura con el identificador 234, la siguiente URI sería la correcta, independientemente de que vayamos a consultarla en formato pdf, xml, txt, xml o json:

/facturas/234

Uso correcto de URIs

Endpoint (punto de acceso a un recurso)

Las URIs deben mantener una jerarquía lógica

- Por ejemplo, la URI **/facturas/234/cliente/007** no sería una URI correcta, ya que no sigue una jerarquía lógica.
- Para el recurso factura con el identificador 234 del cliente 007, la siguiente URI sería la correcta: **/clientes/007/facturas/234**

Uso correcto de URIs

Endpoint (punto de acceso a un recurso)

Filtrados y paginación:

- Para filtrar, ordenar, paginar o buscar información en un recurso, debemos hacer **una consulta sobre la URI, utilizando parámetros HTTP** en lugar de incluirlos en la misma.
- Por ejemplo, la URI **/facturas/orden/desc/fecha-desde/2007/pagina/2** sería incorrecta ya que el recurso de listado de facturas sería el mismo, pero utilizaríamos una URI distinta para filtrarlo, ordenarlo o paginarlo.
- La URI correcta en este caso sería:

/facturas?fecha-desde=2007&orden=DESC&pagina=2

Hipermedia o HATEAOS

Hypermedia as the Engine of Application State

Hipermedios como motor del estado de las aplicaciones

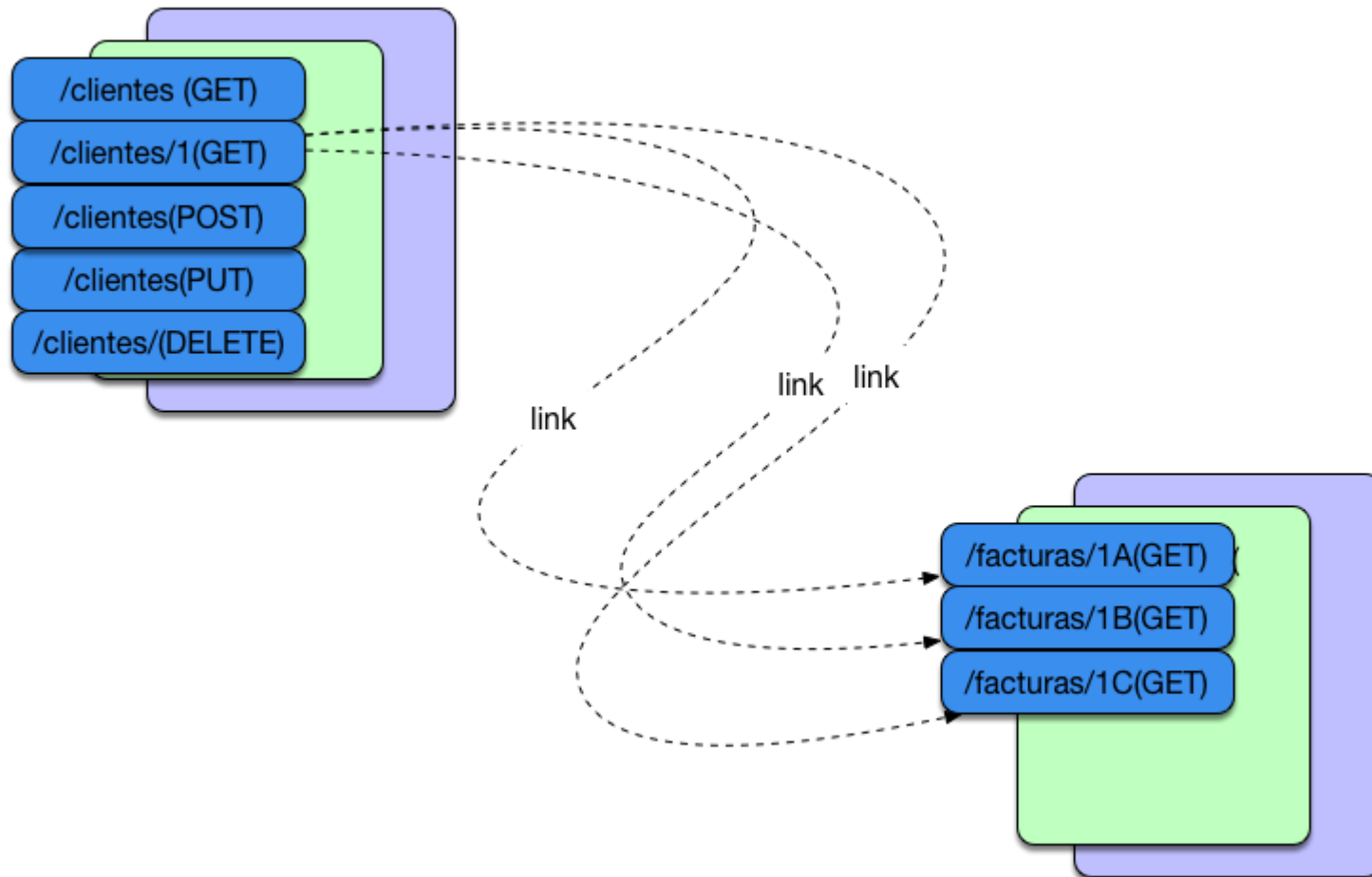
Consiste en conectar mediante vínculos las aplicaciones clientes con las APIs, permitiendo a dichos clientes despreocuparse por conocer de antemano cómo acceder a los recursos.

Con Hipermedia básicamente añadimos información extra al recurso que devolvemos sobre su conexión a otros recursos relacionados con él.

Los Recursos pueden estar relacionados entre ellos **a través del uso de links**

Ejemplo:

Cuando solicitamos la lista de Clientes esta lista puede incluir **links** a las facturas de cada uno de ellos.



Capacidad de almacenamiento en caché

Las cabeceras HTTP comúnmente usadas para esta función:

- **Expires:** especifica hasta cuándo se puede considerar fresca o válida la información.
- **Cache-control:** Es usado para especificar directivas de caché que deben ser respetadas por todos los puntos por los que pasa la información.
- **Vary:** es usada para indicar qué otras cabeceras se deben tener en cuenta para decidir si el servidor puede reutilizar una respuesta o no.
- **Etag:** identifica a una respuesta de manera única, puede usarse para pedir una respuesta nueva de un mismo endpoint, y en combinación con la cabecera If-None-Match

Capacidad de almacenamiento en caché

Las cabeceras HTTP comúnmente enviadas desde php:

```
<?php
header('Cache-Control:public, max-age=600, s-maxage=3600');//HTTP/1.1
header("Expires: ".gmdate("D, d M Y H:i:s", time() + 600)." GMT");
header("Last-Modified: " . gmdate("D, d M Y H:i:s") . " GMT");
?>

//es almacenable en caché por cachés locales y compartidas(proxis)
//es almacenable en caché durante 10 minutos para las cachés locales
//es almacenable en caché durante 1 hora para las cachés compartidas
//Es válida durante 10 minutos
//Le indicamos la fecha de la última modificación
```

CORS

- CORS son las siglas de "Cross-origin resource sharing" y es básicamente una restricción de acceso a recursos que están localizados en otros dominios.
- Tu página o aplicación (JavaScript) puede estar en <http://midominio.com> y tu servidor de API (PHP) puede estar en <http://otrodominio.com>
- Por las restricciones de CORS, por motivos relacionados con la seguridad, no se puede acceder a recursos (APIs) desde clientes(Javascript) que se encuentren en otros dominios.

CORS

Para evitar que la barrera de CORS nos afecte, el servidor tiene que emitir unas cabeceras HTTP en la respuesta.

Estas cabeceras le dicen al navegador que ciertos recursos sí que van a estar disponibles desde otros dominios distintos del habitual.

Así que desde PHP es tan sencillo como escribir unas pocas líneas de código para el envío de esas cabeceras y nuestra API estará disponible para el acceso desde otros dominios.

Las cabeceras son las siguientes.

```
header('Access-Control-Allow-Origin: *');  
header("Access-Control-Allow-Headers: Origin, X-Requested-With, Content-Type, Accept");  
header('Access-Control-Allow-Methods: GET, POST, PUT, OPTIONS, DELETE');  
header("Allow: GET, POST, OPTIONS, PUT, DELETE");
```

CORS

Access-Control-Allow-Origin

- En la primera cabecera estamos diciendo que se permita el acceso a los recursos desde cualquier origen (cualquier dominio, aunque se podría especificar sobre qué dominios en particular, si es que solo queremos permitir el acceso desde determinados dominios).
- Esta primera cabecera es la más importante y la que primeramente percibirás que te está faltando. En su ausencia, el navegador te marcará el siguiente error: "Response to preflight request doesn't pass access control check: No 'Access-Control-Allow-Origin' header is present on the requested resource."

Access-Control-Allow-Headers

- La segunda dice que se acepten diversos content-type, como el "application/json" que usan los servidores habitualmente.
- Esta cabecera también es fundamental y si no la colocas el navegador te lo marcará con el error: "Request header field content-type is not allowed by Access-Control-Allow-Headers in preflight response"

Access-Control-Allow-Methods

- La tercera dice que se permitan diversos métodos de conexión que se suelen usar la comunicación con el servicio web, para las distintas operaciones del API. Estos son GET, POST, PUT, **OPTIONS** y DELETE.
- Esta tercera cabecera en principio parece que no es necesaria, ya que el servidor permite el acceso con métodos como POST de manera predeterminada, pero nunca está de más indicarlo en tu API.

POSTMAN

Postman es una plataforma de colaboración para el desarrollo de **API**.

Postman permite a los usuarios diseñar, simular, depurar, probar, documentar, supervisar y publicar **API**

<https://www.postman.com/downloads/>

A close-up photograph of several sharpened pencils of various colors (blue, green, yellow, pink, red) pointing their tips towards a central point. The pencils are arranged in a radial pattern, creating a starburst effect. The background is dark and out of focus.

`$_SERVER`
Información del entorno del servidor y de ejecución

`$_SERVER['SERVER_NAME']`

El nombre del servidor anfitrión bajo el que está siendo ejecutado el script actual

`$_SERVER['SERVER_PORT']`

Indica el número de puerto en el servidor web que está siendo utilizado.

`$_SERVER['HTTP_HOST']`

Muestra el nombre del host en el encabezado HTTP de la solicitud. Puede incluir el número de puerto si el cliente lo especifica en la solicitud.

`$_SERVER['REQUEST_METHOD']`

Método de petición empleado para acceder a la página, por ejemplo 'GET', 'POST', 'PUT', 'DELETE'

`$_SERVER['SCRIPT_NAME']`

La ruta absoluta del nombre del script que está siendo ejecutado actualmente.

`$_SERVER['PHP_SELF']`

El nombre del archivo de script ejecutándose actualmente, relativa al directorio raíz de documentos del servidor.

`$_SERVER['QUERY_STRING']`

Contiene la cadena de la consulta de la petición de la página, si existe. Contiene todo lo que va detrás de signo ?

`$_SERVER['PATH_INFO']`

Contiene cualquier información sobre la ruta proporcionada por el cliente a continuación del nombre del fichero del script actual, pero antecediendo a la cadena de la petición, si existe. Ej. Si se accede con URI **`http://www.example.com/php/path_info.php/some/stuff?foo=bar`**

`$_SERVER['PATH_INFO']` contendrá `/some/stuff`

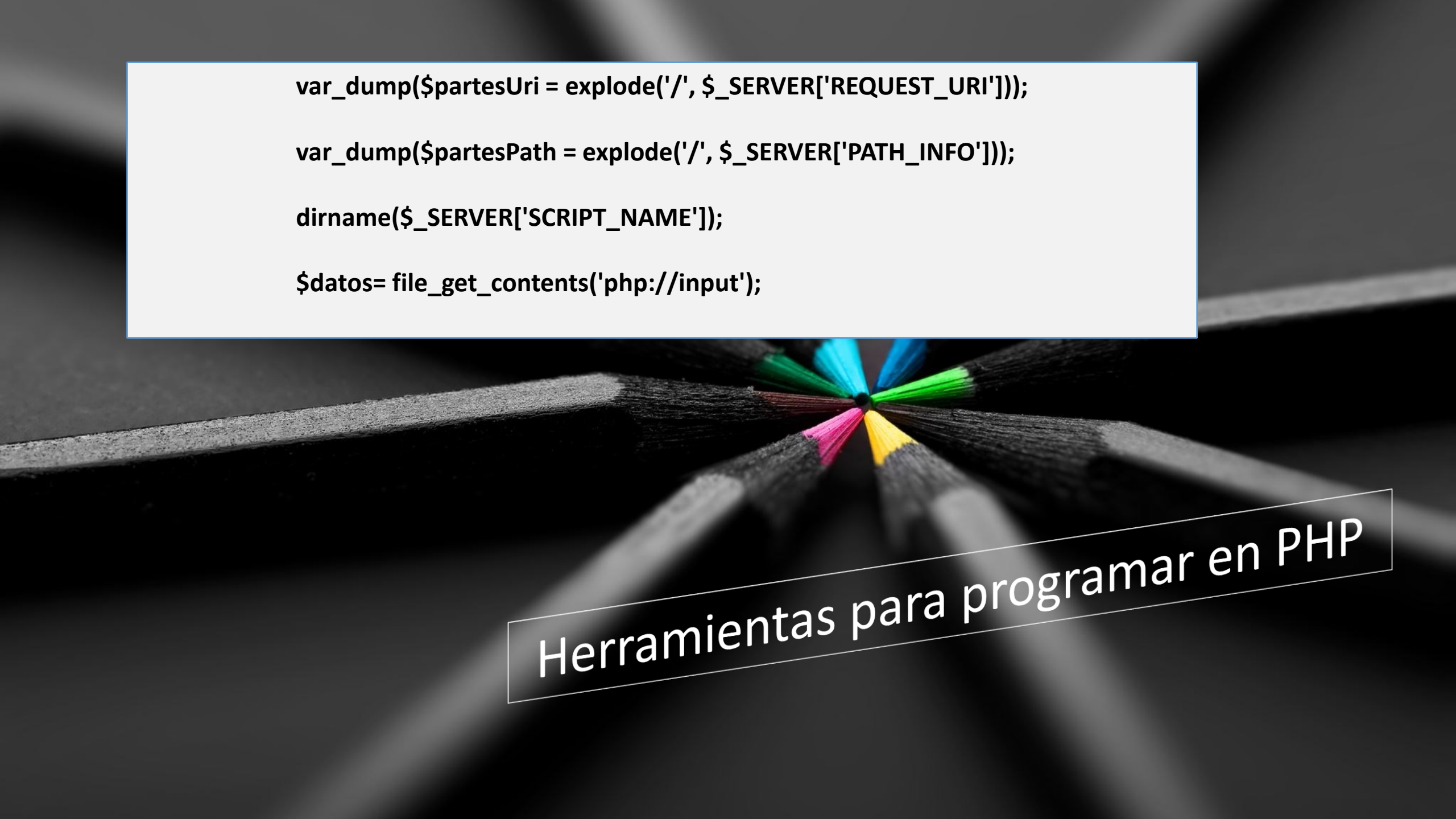
`$_SERVER['REQUEST_URI']`

El URI que fue dado para acceder a la página actual.

`$_SERVER['DOCUMENT_ROOT']`

El directorio raíz de documentos bajo el que está siendo ejecutado el script actual, tal y como se define en el archivo de configuración del servidor web.

```
var_dump($partesUri = explode('/', $_SERVER['REQUEST_URI']));  
  
var_dump($partesPath = explode('/', $_SERVER['PATH_INFO']));  
  
dirname($_SERVER['SCRIPT_NAME']);  
  
$datos= file_get_contents('php://input');
```



Herramientas para programar en PHP